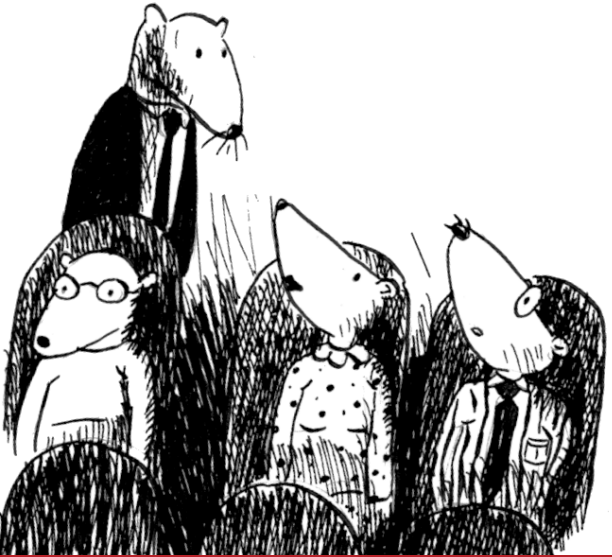


# Capítulo 2 — Sistemas Numéricos

Eduardo Camargo de Siqueira

Tecnologia em Análise e Desenvolvimento de Sistemas

INSTITUTO FEDERAL
Brasília
Campus São Sebastião



drawn by adit.io

Este material foi criado por Eduardo Camargo de Siqueira e está disponível livremente para uso não comercial, para fins didáticos. Se você usar alguma parte deste material, adicione "copyright duca.pro".

Se você encontrar algum erro ou tiver alguma dúvida, envie um e-mail: eduardo.siqueira@ifb.edu.br. Obrigado por contribuir e espero que você aprenda bastante com este material!

# Objetivos do capítulo

Ao final deste capítulo, você deverá ser capaz de:

- compreender os sistemas decimal, binário, octal e hexadecimal;
- converter números entre diferentes bases;
- relacionar números binários com armazenamento e processamento;
- compreender bits, bytes e unidades de medida;
- entender a representação de caracteres, inteiros e números reais.

# Por que computadores usam binário?

Computadores digitais trabalham com circuitos eletrônicos capazes de representar dois estados principais:

- ligado/desligado;
- presença/ausência de tensão;
- verdadeiro/falso;
- 1/0.

**Ideia central**

Toda informação digital precisa ser representada, armazenada e processada como sequências de bits.

# Sistema decimal

O sistema decimal é o mais usado no cotidiano.

- Base 10;
- usa os símbolos: $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$;
- cada posição representa uma potência de 10.

$$347 = 3 \times 10^2 + 4 \times 10^1 + 7 \times 10^0$$

$$347 = 300 + 40 + 7$$

**Sistema posicional**

O valor de um algarismo depende da posição que ele ocupa.

# Decimal e computação

Embora o computador trabalhe internamente com binário, o sistema decimal continua importante.

- interfaces com usuários;
- sistemas financeiros;
- relatórios;
- formulários;
- planilhas;
- bancos de dados;
- aplicações comerciais.

**Exemplo**

O usuário vê R$ 1.250,00, mas o computador armazena e processa esse valor em bits.

# Sistema binário

O sistema binário é o sistema de base 2.

- usa apenas os símbolos 0 e 1;
- cada dígito binário é chamado de bit;
- cada posição representa uma potência de 2.

$$1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$1011_2 = 8 + 0 + 2 + 1 = 11_{10}$$

# Primeiros números em binário

| Decimal | Binário | Decimal | Binário |
|---|---|---|---|
| 0 | 0 | 8 | 1000 |
| 1 | 1 | 9 | 1001 |
| 2 | 10 | 10 | 1010 |
| 3 | 11 | 11 | 1011 |
| 4 | 100 | 12 | 1100 |
| 5 | 101 | 13 | 1101 |
| 6 | 110 | 14 | 1110 |
| 7 | 111 | 15 | 1111 |

# Bits e combinações

Um bit representa dois valores possíveis:

$$0 \quad \text{ou} \quad 1$$

Com $n$ bits, podemos representar:

$$2^n$$

combinações diferentes.

**Examples**

$$2^8 = 256$$

Um byte, formado por 8 bits, pode representar 256 combinações.

# O binário e os computadores

Textos, imagens, sons, vídeos, programas e instruções são codificados como sequências de bits.

- caracteres recebem códigos numéricos;
- imagens são formadas por pixels;
- sons são transformados em amostras numéricas;
- programas são instruções codificadas.

$$A = 65_{10} = 01000001_2$$

# Sistema octal

O sistema octal é o sistema de base 8.

- usa os símbolos: $0, 1, 2, 3, 4, 5, 6, 7$;
- cada posição representa uma potência de 8;
- tem relação direta com o binário.

$$157_8 = 1 \times 8^2 + 5 \times 8^1 + 7 \times 8^0$$

$$157_8 = 64 + 40 + 7 = 111_{10}$$

# Octal e binário

Como:

$$8 = 2^3$$

cada dígito octal corresponde a 3 bits.

| Octal | Binário |
|---|---|
| 0 | 000 |
| 1 | 001 |
| 2 | 010 |
| 3 | 011 |
| 4 | 100 |
| 5 | 101 |

# Octal em permissões Linux

Permissões em sistemas Unix/Linux podem ser representadas em octal.

| Permissão | Símbolo | Valor |
|---|---|---|
| Leitura | r | 4 |
| Escrita | w | 2 |
| Execução | x | 1 |

$$7 = 4 + 2 + 1 = r + w + x$$

**Exemplo**

```
chmod 755 script.sh
```

# Sistema hexadecimal

O sistema hexadecimal é o sistema de base 16.

- usa os símbolos 0 a 9;
- usa as letras $A$ a $F$;
- representa binários longos de forma compacta.

$$A = 10, \quad B = 11, \quad C = 12, \quad D = 13, \quad E = 14, \quad F = 15$$

# Hexadecimal: exemplo

$$2F_{16} = 2 \times 16^1 + F \times 16^0$$

Como:

$$F = 15$$

temos:

$$2F_{16} = 2 \times 16 + 15 \times 1$$

$$2F_{16} = 32 + 15 = 47_{10}$$

# Hexadecimal e binário

Como:

$$16 = 2^4$$

cada dígito hexadecimal corresponde a 4 bits.

| Hex | Binário | Hex | Binário |
|---|---|---|---|
| 0 | 0000 | 8 | 1000 |
| 1 | 0001 | 9 | 1001 |
| 2 | 0010 | A | 1010 |
| 3 | 0011 | B | 1011 |
| 4 | 0100 | C | 1100 |
| 5 | 0101 | D | 1101 |

# Hexadecimal em cores digitais

Em HTML e CSS, cores são frequentemente representadas em hexadecimal.

$$\#RRGGBB$$

- `RR`: vermelho;
- `GG`: verde;
- `BB`: azul.

**Exemplos**

$$\#FFFFFF = \text{branco}$$

$$\#000000 = \text{preto}$$

# Por que hexadecimal é tão usado?

Números binários podem ficar longos.

$$11111111_2$$

Agrupando em blocos de 4 bits:

$$1111\ 1111$$

Cada bloco `1111` corresponde a `F`.

$$11111111_2 = FF_{16}$$

**Ideia**

Hexadecimal é uma forma compacta de escrever bytes e valores binários.

# Conversão entre bases

Converter entre bases significa representar a mesma quantidade de outra forma.

$$10_{10} = 1010_2 = 12_8 = A_{16}$$

**Atenção**

A quantidade não muda. O que muda é a representação.

# Decimal para binário

Método: divisões sucessivas por 2.

$$25 \div 2 = 12 \quad r = 1$$

$$12 \div 2 = 6 \quad r = 0$$

$$6 \div 2 = 3 \quad r = 0$$

$$3 \div 2 = 1 \quad r = 1$$

$$1 \div 2 = 0 \quad r = 1$$

Lendo de baixo para cima:

# Binário para decimal

Multiplicamos cada dígito pela potência de 2 correspondente.

$$11001_2 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$11001_2 = 16 + 8 + 0 + 0 + 1$$

$$11001_2 = 25_{10}$$

# Conversões com octal e hexadecimal

**Decimal para octal**

$$95_{10} = 137_8$$

Método:

- divisões sucessivas por 8;
- leitura dos restos de baixo para cima.

**Decimal para hexadecimal**

$$254_{10} = FE_{16}$$

Método:

- divisões sucessivas por 16;
- restos 10 a 15 viram A a F.

# Conversão direta

- Binário para octal: agrupar bits de 3 em 3.
- Octal para binário: substituir cada dígito por 3 bits.
- Binário para hexadecimal: agrupar bits de 4 em 4.
- Hexadecimal para binário: substituir cada dígito por 4 bits.

**Exemplo**

$$11110110_2 = 1111\ 0110 = F6_{16}$$

# Erros comuns em conversões

- ler restos na ordem errada;
- confundir valor com representação;
- esquecer letras A a F no hexadecimal;
- agrupar bits pelo lado errado;
- remover zeros à esquerda quando eles indicam tamanho fixo.

**Exemplo**

$$00101_2 = 101_2$$

mas em um byte pode ser necessário escrever:

$$00000101_2$$

# Conversões em Python

## Decimal para outras bases

```
numero = 254

print(bin(numero))  # 0b11111110
print(oct(numero))  # 0o376
print(hex(numero))  # 0xfe
```

## Outras bases para decimal

```
print(int("11111110", 2))
print(int("376", 8))
print(int("fe", 16))
```

# Bits, bytes e unidades de medida

- 1 bit: menor unidade de informação;
- 1 byte: 8 bits;
- 1 KB: aproximadamente mil bytes;
- 1 MB: aproximadamente um milhão de bytes;
- 1 GB: aproximadamente um bilhão de bytes;
- 1 TB: aproximadamente um trilhão de bytes.

**Importância**

Tamanho de arquivos, memória e armazenamento são medidos a partir dessas unidades.

# Unidades decimais e binárias

Há diferença entre unidades decimais e binárias.

- $1\,KB = 1.000$ bytes;
- $1\,KiB = 1.024$ bytes;
- $1\,MB = 1.000.000$ bytes;
- $1\,MiB = 1.048.576$ bytes.

**Por quê?**

Computadores trabalham naturalmente com potências de 2.

# Representação de caracteres

Computadores também precisam representar textos.

- letras;
- acentos;
- números;
- símbolos;
- espaços;
- emojis.

**Ideia central**

Cada caractere é associado a um código numérico, armazenado em bits.

# ASCII

ASCII é uma tabela histórica de codificação de caracteres.

- usa originalmente 7 bits;
- representa 128 valores;
- inclui letras do inglês, números e sinais de pontuação;
- não representa bem acentos e outros alfabetos.

$$A = 65_{10} = 01000001_2$$

# Unicode e UTF-8

**Unicode** define códigos para caracteres de vários idiomas.

$$A = \texttt{U+0041}$$

$$ç = \texttt{U+00E7}$$

**UTF-8** define como esses códigos são armazenados em bytes.

- caracteres simples: 1 byte;
- acentos: geralmente mais de 1 byte;
- emojis: frequentemente 4 bytes.

# Caracteres versus bytes

A quantidade de caracteres visíveis nem sempre é igual à quantidade de bytes.

- casa: 4 caracteres, geralmente 4 bytes em UTF-8;
- ação: 4 caracteres, mas pode ocupar mais de 4 bytes;
- emojis podem ocupar ainda mais espaço.

**Aplicações**

Arquivos CSV, páginas web, APIs, bancos de dados e sistemas multilíngues.

# Caracteres em HTML e Python

## HTML

```
<meta charset="UTF-8">
```

## Python

```
print(ord("A"))
print(ord("ç"))
print(chr(65))
print(chr(231))

texto = "ação"
print(len(texto))
print(len(texto.encode("utf-8")))
```

# Representação de números inteiros

Inteiros são números sem parte fracionária.

$$-3,\ -2,\ -1,\ 0,\ 1,\ 2,\ 3$$

Em programação, aparecem em:

- contadores;
- índices;
- identificadores;
- permissões;
- códigos de status;
- tamanhos de arquivos;
- endereços.

# Inteiros sem sinal

Com $n$ bits:

$$2^n$$

combinações possíveis.

Para inteiros sem sinal:

$$0 \quad \text{até} \quad 2^n - 1$$

**Exemplo**

Com 8 bits:

0 até 255

# Inteiros com sinal

Inteiros com sinal representam valores positivos e negativos.
Em complemento de dois, com $n$ bits:

$$-2^{n-1} \quad \text{até} \quad 2^{n-1} - 1$$

**Exemplo**

Com 8 bits:

$$-128 \quad \text{até} \quad 127$$

# Complemento de dois

Para representar o negativo de um número:

1. escreva o número positivo em binário;
2. inverta todos os bits;
3. some 1.

**Exemplo: $-5$ em 4 bits**

$$5 = 0101$$

$$0101 \rightarrow 1010$$

$$1010 + 1 = 1011$$

# Overflow

Overflow ocorre quando o resultado não cabe na quantidade de bits disponível.

## Exemplo com 4 bits sem sinal

$$1111_2 = 15_{10}$$

$$15 + 1 = 10000_2$$

O resultado exige 5 bits.

## Atenção

Overflow pode produzir resultados inesperados em programas.

# Números com parte fracionária

Computadores também precisam representar valores como:

$$3{,}14, \quad -0{,}5, \quad 0{,}001, \quad 1500{,}75$$

Eles aparecem em:

- medições;
- valores monetários;
- porcentagens;
- sensores;
- estatística;
- jogos;
- inteligência artificial.

# Frações em binário

No binário, casas à direita da vírgula representam potências negativas de 2.

$$101{,}101_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$$

$$101{,}101_2 = 4 + 1 + 0{,}5 + 0{,}125$$

$$101{,}101_2 = 5{,}625_{10}$$

# Ponto flutuante

A representação de ponto flutuante é semelhante à notação científica.

$$\text{número} = \text{sinal} \times \text{mantissa} \times 2^{\text{expoente}}$$

Ela permite representar:

- números muito grandes;
- números muito pequenos;
- valores aproximados com quantidade finita de bits.

# Precisão e arredondamento

Nem todo número decimal tem representação binária finita.

## Exemplo clássico

```
print(0.1 + 0.2)
```

Saída possível:

```
0.30000000000000004
```

## Atenção

Isso não é um erro da linguagem: é consequência da representação finita em binário.

# Dinheiro e números decimais

Em sistemas financeiros, pequenos erros de arredondamento podem causar problemas. Estratégias comuns:

- armazenar dinheiro como inteiro em centavos;
- usar tipos decimais próprios;
- evitar comparação direta entre pontos flutuantes.

$$R\$\,19{,}99 \rightarrow 1999 \text{ centavos}$$

# Síntese do capítulo

- sistemas numéricos são formas diferentes de representar valores;
- binário é a base da computação digital;
- octal e hexadecimal compactam representações binárias;
- caracteres são códigos numéricos armazenados em bits;
- inteiros dependem da quantidade de bits e do sinal;
- números reais podem ser aproximados em ponto flutuante.

# Atividade prática

Converta os seguintes valores:

1. $37_{10}$ para binário, octal e hexadecimal;
2. $101101_2$ para decimal, octal e hexadecimal;
3. $72_8$ para decimal, binário e hexadecimal;
4. $2A_{16}$ para decimal, binário e octal.

**Reflexão**

Em quais conversões o binário funcionou como ponte entre as bases?

# Perguntas para reflexão

- Converter um número para outra base muda o valor ou apenas sua representação?
- Por que zeros à esquerda podem ser irrelevantes em matemática, mas importantes em computação?
- Como problemas de codificação podem afetar sistemas reais?
- Por que overflow é importante para desenvolvedores?
- Por que números decimais exigem cuidado em aplicações financeiras?

# FIM

- Dúvidas!?
- http://duca.pro.br/ — eduardo.siqueira@ifb.edu.br
- Ilustrações: copyright Manning Publications, drawn by adit.io
- Obrigado pela atenção!